

SOA

Service Oriented Architecture

CMP 7174

Introduction to SOA Enabling Standards & Technologies

Dr. AbdulRahman Alsewari

rahman.alsewari@bcu.ac.uk

Contract

Implementation

Interface

Department of Computer Science
School of ABCE
Birmingham City University

Business logic

Data

Application
frontend

Service

Service repository

Service bus

Assoc. Prof. Dr AbdulRahman Alsewari

Deputy Course Lead BSc Computer Science with Artificial Intelligence

Email: rahman.alsewari@bcu.ac.uk

- PhD, University Science Malaysia 2012.
- Assistant Professor, Head of IT dept. Darassalam University, Yemen, 2012.
- Senior Lecturer, University Malaysia Pahang, Malaysia, 2013-2020, Malaysia.
- Assoc. Prof ,University Malaysia Pahang, Malaysia, 2020-2022, Malaysia.
- Visiting Professor, Al-Bayan University, Iraq, 2024.
- Editor, Guest editor, Reviewer for several journals.
- General Chair/TCM of several international conferences. .
- Associate Professor, Birmingham City University, UK, 2022.
 - Course Lead BSc Computer Science with Artificial Intelligence
 - International Summer school for AI & ML 2024-2025

Research Interests:

- Software Engineering, Software Testing
- Artificial Intelligence, Optimisation Algorithms,
- Deep learning and Machine learning
- Image Processing



<https://alsewari.org/>

<https://scholar.google.com/citations?user=hLypPv8AAAAJ&hl=en>

<https://www.scopus.com/authid/detail.uri?authorId=57215983138>



Please write your:

- 1. Bachelor's degree**
- 2. Software and Applications development experience**
- 3. Experience with APIs**
- 4. Experience with Microservices**
- 5. Programming languages you know**

Module Overview - CMP 7174

Module Aim

In-depth knowledge of **service oriented system design** principles

Understanding of **basic technologies** for SOA realisation

Skills in service interface specification in WSDL, REST

Implementation and deployment of atomic services

Assessment

Group Coursework Assignment and Individual

Design document (20%) - Week commencing 30th Oct 2026

Executable code (50%) - Week commencing 15th Dec 2026

Presentation & demo (30%) - Week commencing commencing 15th Dec 2026

Content & Delivery

Lectures: Wk1 – Wk12 (abstract level discussion)

Lab Sessions: Wk1 – Wk12 (practical work)

Self Study: 3-4 hours/week

Materials available via BCU Moodle

Resources

Papazoglou, "Web Services and SOA: Principles and Technology"

Erl, "Service-Oriented Architecture: Concepts, Technology, and Design"

Computing Labs with necessary software

Additional materials via BCU Moodle

Technologies & Tools - CMP 7174

- Visual Studio 2022 with ASP.NET and WCF
- IntelliJ IDEA
- VSCode - Codespace in Github
- MongoDB
- SSMS
- Postman
- SOAPUI
- Camunda
- Azure
- UML, BPMN, Lucidchart



Please type your understanding of the assessment plan. In your answer, include:



- 1. The number of submissions:**
- 2. Whether they are individual or group-based:**
- 3. The weightage of each submission:**
- 4. The submission dates:**
- 5. The number of members in a group (if applicable):**
- 6. What should be covered in each submission?**

TEACHING WEEK	LECTURE TOPIC	TUTORIAL / LAB TOPIC
1	Introduction to Web Services and Service Oriented Architecture	Discuss the assessments. Introduce the technologies and tools that will be used during this semester. Setup the tutorial environment.
2	Introduction to SOA Enabling Standards & Technologies (SOAP & WSDL), Web Services Platforms: .NET	Implement a WCF to show WSDL & SOAP. Exercises on SOAP & WSDL Implementation and Consuming Web Services in .NET
3	Design of Service Workflows SOA Life Cycle	Design Workflows Using UML diagrams and BPMN (Camunda)
4	Introduction to Microservices Arch using .NET	Implementation Microservices in .NET for Purchase system (Order Service). Using MongoDB database
5	Microservices Arch using .NET	Implementation two Microservices in .NET for Purchase system (Customer Service). Using SSMS database
6	Microservices Arch using Java	Implementation two Microservices in .java for Purchase system (Product Service). Using MongoDB database
7	Services Orchestration by Api gateway	Implement the API Gateway microservice, and the UI
8	Inventory Microservice using Python and the FastAPI framework	Tutorial on Building an Inventory Microservice using Python and the FastAPI framework
9	GitHub for Service-Oriented Architecture Development	Tutorial on GitHub for Service-Oriented Architecture Development
10	Deploying Microservices to the Cloud (Render / Microsoft Azure)	Tutorial on Deploying Microservices to the Cloud (Render / Microsoft Azure)
11	Assessment preparation	Assessment preparation
12	Assessment preparation	Assessment preparation
	Check the Assessment Section	Application and Presentation Submission

Introduction to Web Services and SOA

🔑 What is Software Architecture?

Three Main Types:

Monolithic Architecture - Single, unified unit

Service-Oriented Architecture (SOA) - Interoperable services

Microservices Architecture - Fine-grained services

🕒 Origins of SOA

Evolved from business need to interact and use each others' services

Businesses collaborate and compete in service-oriented environment

Provides well-defined, loosely coupled interaction between services

↔ Service-Oriented Paradigm

Utilizes services as constructs for rapid, low-cost application composition

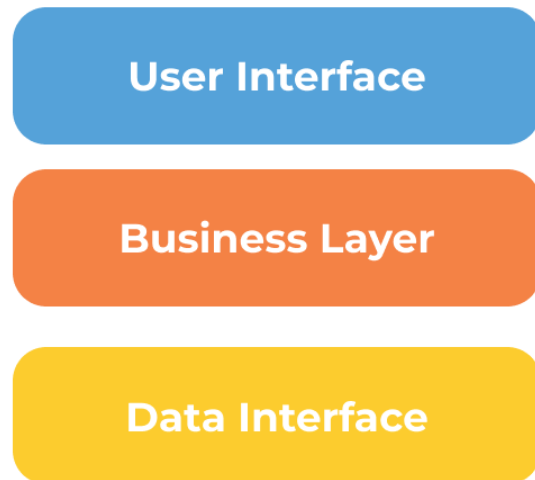
Web Service - Programmatically available application logic over Internet

Composing applications by discovering and invoking network-available services

⚙ What are Web Services?

Encapsulated business functions: Self-contained business tasks Full-fledged business processes Service-enabled applications Enable interaction between distributed systems

Monolithic

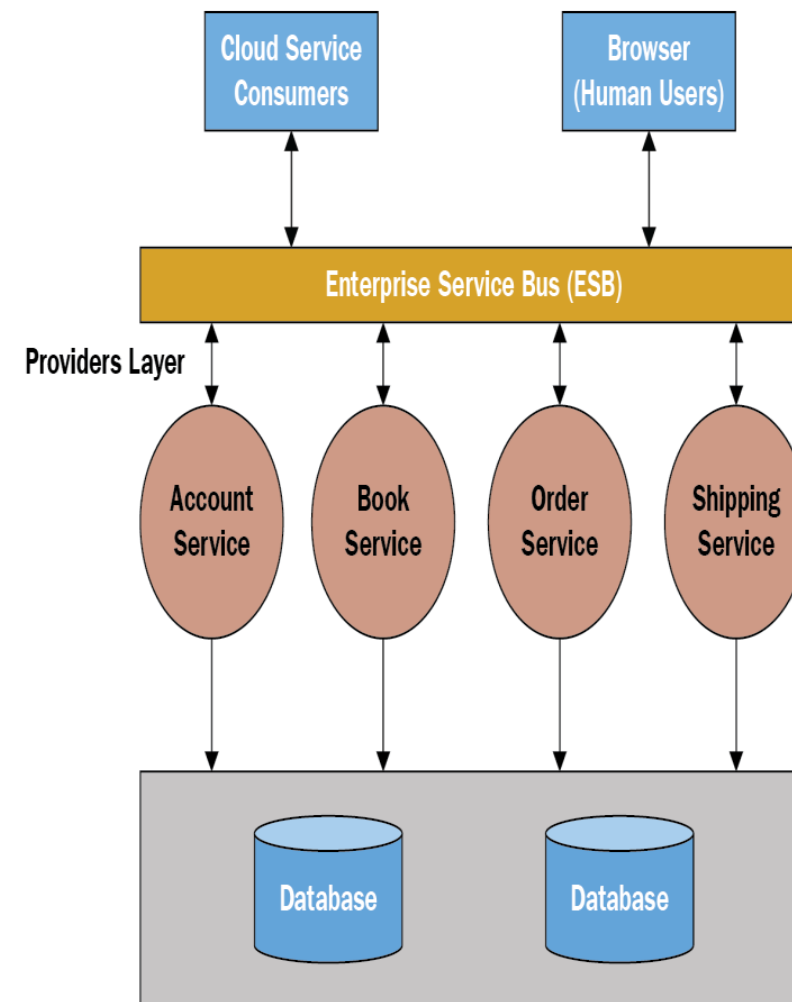


Monolith: Single Database

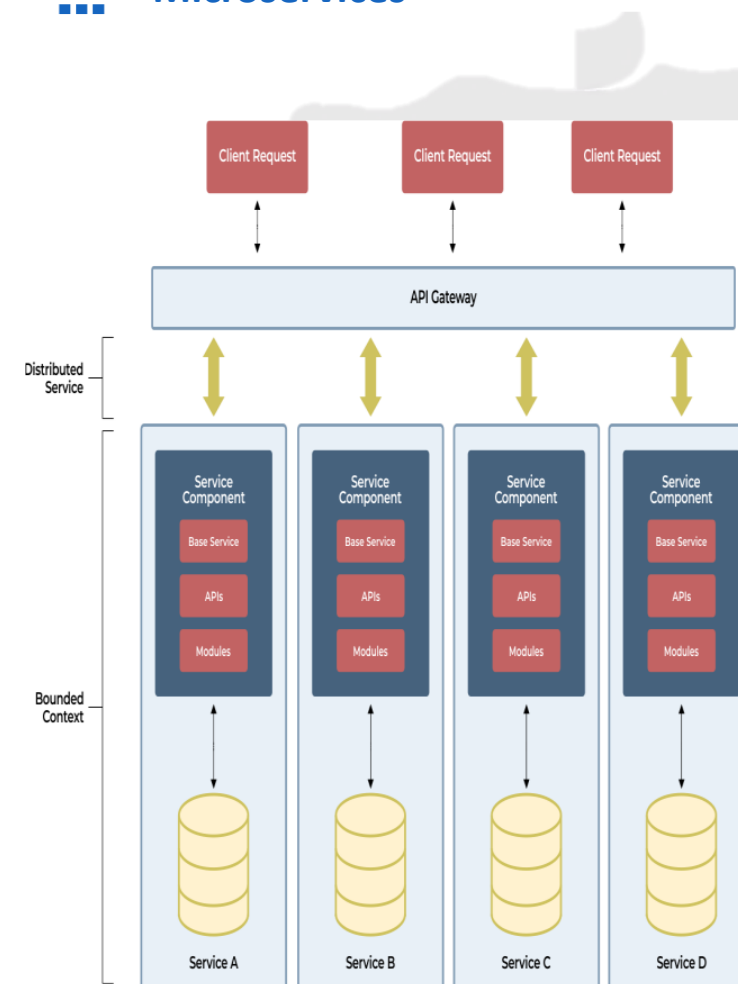


Service-Oriented (SOA)

Consumers Layer



Microservices





Which architecture type is characterized by a single, unified deployment unit?



Which architecture type is best suited for independent scaling of components?

Origins of SOA

🕒 Evolution of Service Architecture

Business-Driven Development

Evolved from business need to **interact and use** each others' services

Businesses collaborate and compete in service-oriented environment

Provides **well-defined, loosely coupled** interaction between services

Enables businesses to become 'service oriented'

🚗 Online Car Rental Business

Allows customers to select cars and car park

Accepts payments

Releases electronic key of the car

💡 Business Services Ecosystem

No Single Service Provider

Multiple businesses provide different services

Each business specializes in specific services

Customers use composite services from multiple providers

Services must interoperate seamlessly

Key Services in Car Rental

Vehicle inventory service

Payment processing service

Authentication service

Location tracking service

Customer management service

Service-Oriented Paradigm

↔ Core Concepts

Services as Building Blocks

Utilizes **services** as constructs for rapid application development

Enables **low-cost composition** of distributed applications

Focus on reusing existing services rather than building from scratch

🌐 Web Service Definition

Programmatically available **application logic** exposed over Internet

Any code or component can become a network-available service

🧑 Service-Oriented Approach

Composition over Construction

Applications built by **discovering and invoking** network-available services

Shift from monolithic development to service integration

Emphasis on interoperability and loose coupling

★ Key Characteristics



Discoverable



Composable



Reusable



Interoperable



Loosely Coupled



Which of the following is a characteristic of Web Services?

Application Service Providers vs Web Services



Application Service Providers

Software-as-a-Service Pioneer

"Rents" applications to subscribers

Complete application with UI, workflow, business and data components

Hosts entire application on provider's servers

Limited customization beyond UI appearance

Alternative: Download modules to customer's site on demand

⚠ Limitations

Inability to develop highly interactive applications

Limited customization capabilities

Difficult application integration



Web Services

Modern Service Architecture

Based on **XML standards / JSON**

Loosely coupled asynchronous interactions

Independent of internal client/server technology

Services described in standard description language

Dynamic discovery and inclusion in applications

Programmatic access to service functionality

✓ Advantages

Easier application communication over Internet

High degree of interoperability

Flexible and customizable integration

Types of Web Services



Informational Services

Services of **simple nature** that provide access to content or expose back-end business applications

★ Key Characteristics

↔ Request/Response

📄 Fine-grained

🔗 Simple Operations

🔌 Data Access

🏠 Examples

☁ Weather Reports

💰 Stock Quotes

📰 News Feeds

💰 Financial Info



Complex Services

Services that involve **assembly and invocation** of multiple pre-existing services from diverse enterprises

★ Key Characteristics

🔗 Multi-step

🔗 Coarse-grained

🔗 Workflow-based

🏢 Cross-enterprise

🏠 Examples

📦 Supply Chain

🛒 Order Processing

📦 Inventory Control

🏢 Financial Services



Which type of Web Service involves assembly of multiple pre-existing services?

Service Interface and Implementation



Service Interface

Public Contract

Defines service functionality **visible to external world**

Provides means to access functionality

Describes: Available operations Parameters and data types Access protocols

Enables other software to: Determine what the service does Know how to invoke it Understand expected results

< > Service Implementation

Internal Logic

Realizes a specific service interface

Implementation details are **hidden** from users

Multiple providers can implement same interface

Can use any programming language

Can evolve independently as long as interface remains consistent

→ Client vs Provider Perspectives



Client Perspective

What does it do?

How do I use it?

Where can I find it?

How to represent it
(business requirements)



Provider Perspective

Where to host it

How to publish it

How to build it

How to maintain
and update it

Interface

Public Contract



Implementation

Internal Logic

Exercise: SOA Business Scenario



Analyze the scenario and identify SOA components

Business Scenario: Online Travel Booking System

A travel company wants to create an online booking system that allows customers to:

- Search for flights from multiple airlines
- Book hotel rooms from various hotel chains
- Reserve rental cars from different agencies
- Make payments through various payment processors

The company doesn't own any airlines, hotels, or car rental agencies but partners with service providers who offer these capabilities through web APIs.



Identify Components

Identify the following SOA components in this scenario:

- ✓ **Service Providers** - Who provides the services?
- ✓ **Service Clients** - Who consumes the services?
- ✓ **Service Registry** - How are services discovered?



Map Interactions

Map out the SOA interactions:

- ➔ How does the travel company discover available services?
- ➔ How does the booking system interact with different service providers?
- ➔ What data needs to be exchanged between components?



Design Considerations

Consider these design aspects:

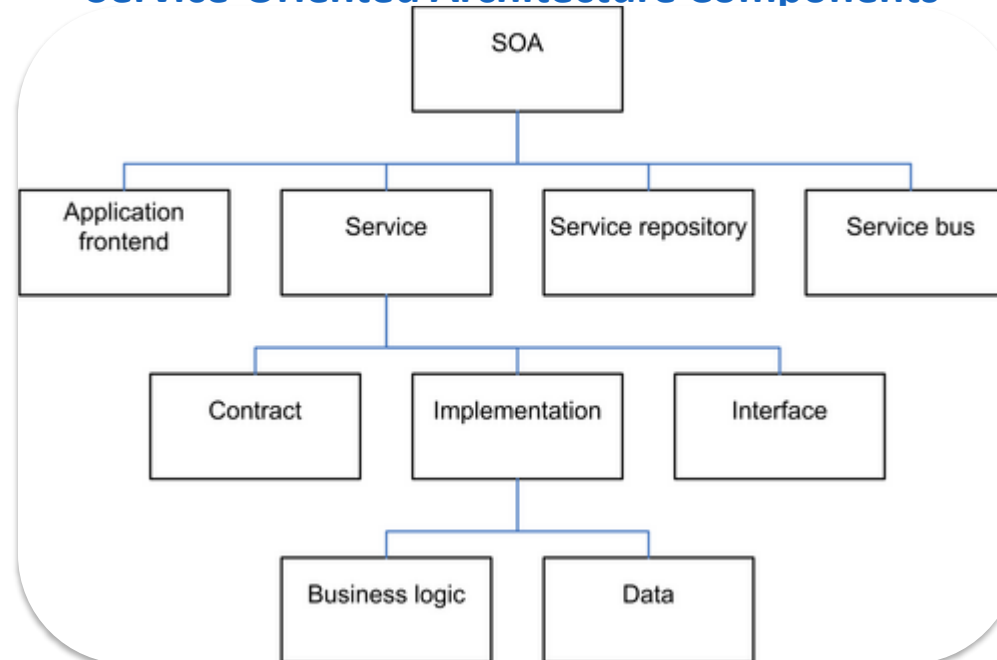
-  **Loose Coupling** - How to ensure components can change independently?
-  **Service Granularity** - Should services be fine or coarse-grained?
-  **Security** - How to secure transactions and data?

Service-Oriented Architecture

What is SOA?

A logical way of designing a software system to provide **services** to either end-user applications or to other services distributed in a network, via **published and discoverable interfaces**.

Service-Oriented Architecture Components



Service Provider

Creates and publishes service descriptions

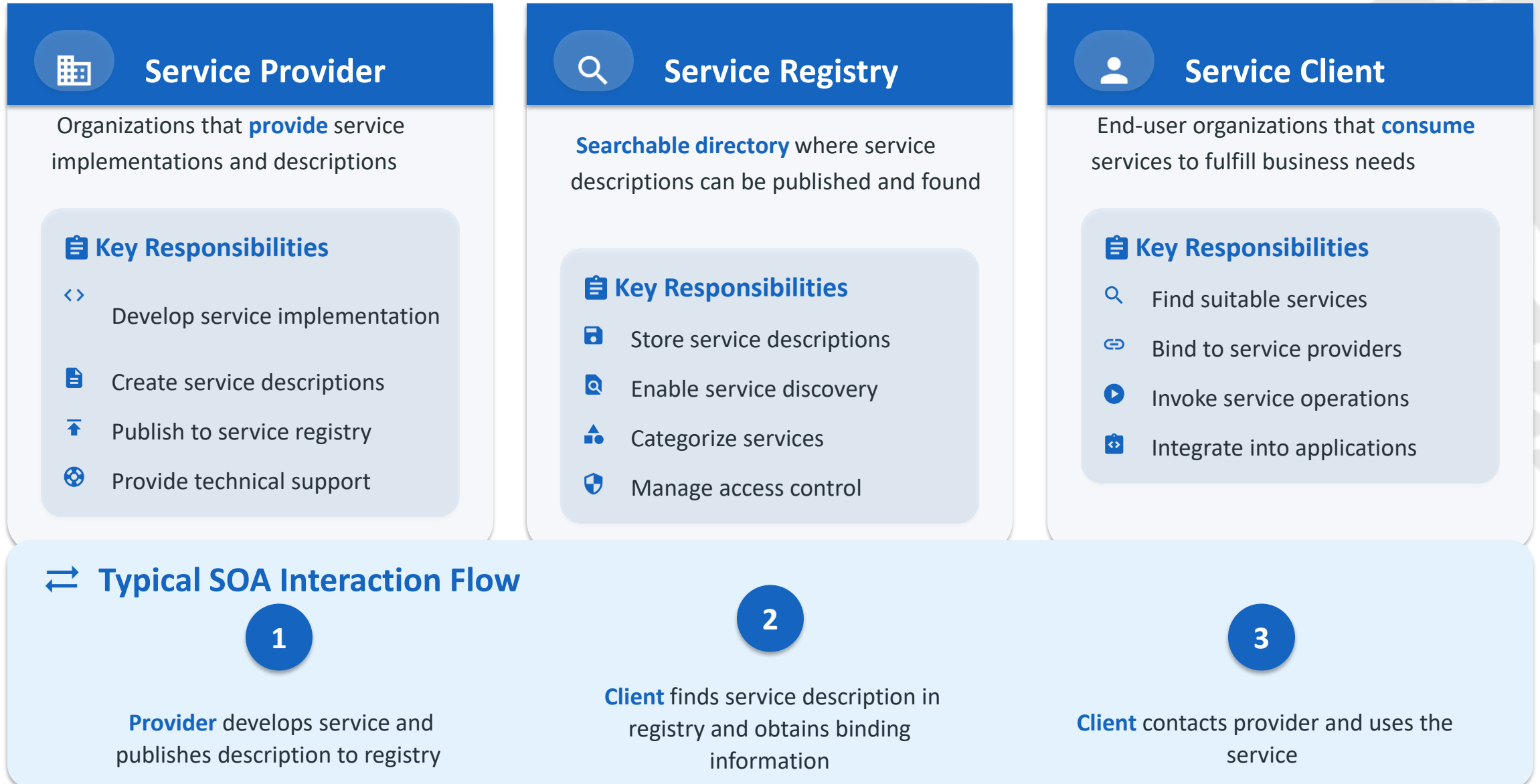
Service Registry

Directory for publishing and finding services

Service Client

Discovers and uses services

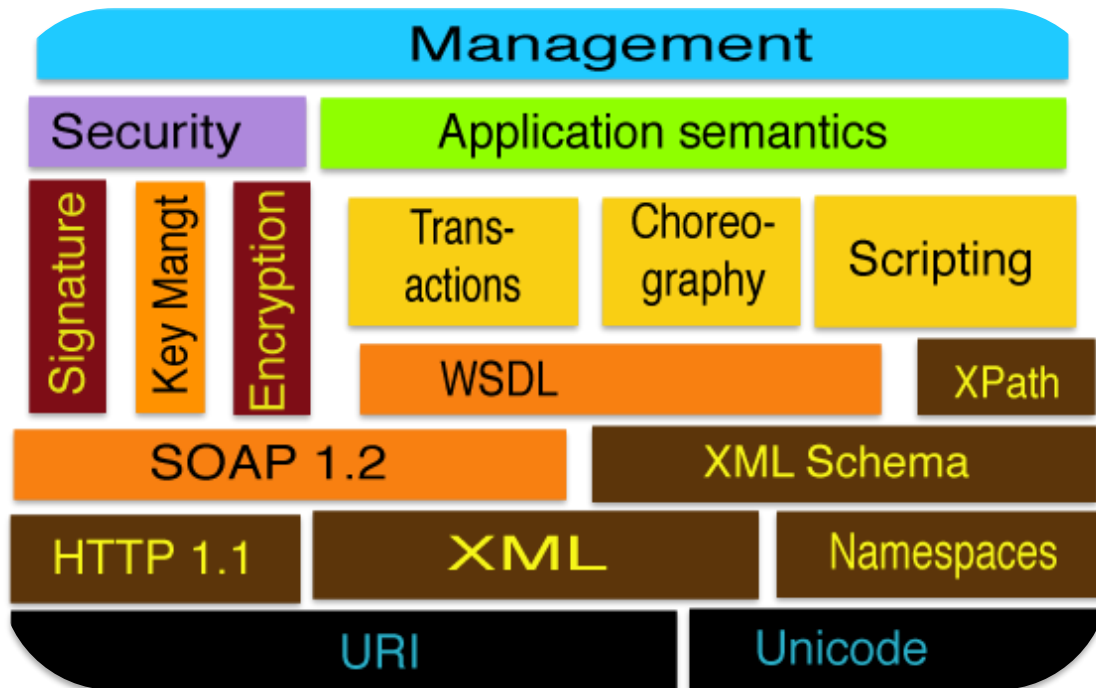
Roles and Operations in SOA



Web Services Technology Stack

Technology Stack Overview

Web services are implemented by a collection of several **related technologies and standards** that work together to enable service-oriented applications.



XML (eXtensible Markup Language)

Foundation for data representation and exchange Standard format for structuring data Platform and language independent



SOAP (Simple Object Access Protocol)

Messaging protocol for information exchange Defines message structure and processing rules Works with various transport protocols

WSDL (Web Services Description Language)

Interface description language XML-based interface definition Describes service functionality and access methods

REST-based SOA

REST Principles

Representational State Transfer



Client-Server

Separation of concerns between client and server



Stateless

No client context stored on server



Cacheable

Responses must explicitly state if cacheable



Uniform Interface

Standardized interface between components



REST in SOA

Modern SOA Implementation

-  **Performance** - Lightweight communication with minimal overhead
-  **Multi-platform** - Works across different devices and platforms
-  **Simplicity** - Easy to implement and understand
-  **Scalability** - Stateless nature supports horizontal scaling

HTTP HTTP Methods in REST



POST

Create new resources



GET

Retrieve resources



PUT

Update existing resources



DELETE

Remove resources

Microservices Architecture

What are Microservices?

Fine-grained Service Architecture



Small & Focused

Single responsibility, independently deployable



Loosely Coupled

Minimal dependencies between services



Technology Diversity

Different tech stacks per service



Decentralized Data

Each service manages its own data

Key Characteristics



Rapid Deployment

Continuous delivery and deployment



Horizontal Scaling

Scale individual services independently



Fault Isolation

Failure in one service doesn't affect others



Team Autonomy

Cross-functional teams own services



Benefits & Challenges



Flexibility - Technology choice per service



Resilience - Failure isolation prevents system-wide crashes



Scalability - Scale only what needs scaling



Complexity - Distributed systems are harder to manage



Testing - End-to-end testing more challenging



Data Consistency - Requires distributed transactions

JSON in Modern SOA



JSON Features

Lightweight Data Interchange Format



Syntax Simplicity

Easy to read and write for humans and machines



Data Structures

Objects, arrays, strings, numbers, booleans, null



Language Agnostic

Parsing support in most programming languages



Native JavaScript

Built-in support in web browsers



JSON in RESTful Services

```
// REST API Response Example { "id": "12345", "name": "Product Service", "endpoints": [
{ "path": "/products", "method": "GET", "description": "Get all products" }, { "path":
"/products/{id}", "method": "GET", "description": "Get product by ID" } ], "status":
"active" }
```



Seamless integration with JavaScript frameworks



Reduced payload size compared to XML



Advantages in Web Development



Performance - Faster parsing and smaller data footprint



Mobile-friendly - Reduced bandwidth usage



Developer experience - Easier to work with in JavaScript



Security - No executable code, reducing XSS risks



Stateless - Perfect for RESTful architectures



JSON Replacing XML



Web APIs

Default format for modern REST APIs and microservices



Mobile Apps

Preferred for mobile app backend communication



Configuration

Increasingly used for application configuration files



NoSQL Databases

Native format for document databases like MongoDB

Industry Trend



90% of modern APIs use JSON as primary data format



Declining use of SOAP/XML in new projects

XML Document Structure

< > What is XML?

eXtensible Markup Language

A **meta-language** that allows creating and using tag sets in a standard way

Provides **fixed rules** about markup structures (elements, attributes, entities)

Enables structured data representation and exchange

Platform and language independent



Document Type: XML documents have formally defined types



Named Containers: Elements, attributes, and declarations



Instance: Represents one possible set of data for a markup language



XML Example

```
<?xml version="1.0" encoding="UTF-8"?> <BillingInformation customer-  
type="manufacturer"> <Name>Right Plastic Products</Name> <BillingDate>2002-  
09-15</BillingDate> <Address> <Street>158 Edward st.</Street>  
<City>Brisbane</City> <State>QLD</State> <PostalCode>4000</PostalCode>  
</Address> </BillingInformation>
```

Key Components

Elements: Fundamental units of content (e.g., <Name>)

Attributes: Name-value pairs in start tags (e.g., customer-type)

Root Element: Topmost element containing all others

Nested Elements: Parent-child relationships



What is the purpose of XML namespaces?



Which of the following is a characteristic of a simple element in XML Schema?



What is the primary purpose of XML Schema (XSD)?

Summary and Conclusion



Service-Oriented Architecture



Loosely coupled services that interact over networks



Service-oriented paradigm for rapid application composition



Three key roles: **Service Provider**, **Service Client**, and **Service Registry**



Technology stack includes **XML**, **JSON**, **SOAP**, **REST**, **WSDL**, and **UDDI**

Key Takeaway

SOA and XML work together to enable interoperable, loosely coupled systems with standardized data exchange